



WASDCAT Games
INDEPENDENT GAME STUDIO

Brand Kit Generator

Creating transparent overlays for social media

VERSION

1.2.0

AUTHOR

Frank Winter

STATUS

Released

DATE

2026-09-20

Table of Contents

Getting started	4
1 What this tool does	5
1.1 The five building blocks	5
1.2 The four formats	6
1.3 What you need	6
1.4 Language	6
1.5 Where your data lives	7
2 First steps	8
2.1 Starting the application	8
2.2 Your first post in five steps	8
2.3 What to set up next	9
3 The interface	10
3.1 Header	10
3.2 Sidebar	10
3.3 Preview	11
3.4 Export bar	12
Working	13
4 Creating a post	14
4.1 Selection	14
4.2 Content	14
4.3 The preview keeps up	15
5 Templates	16
5.1 The templates that ship	16
5.2 Placeholders	16
5.3 Mapping the typography	17
5.4 Taking a template out of the catalogue	17
5.5 Adding your own template	18
6 Colour sets	19
6.1 The seven roles	19
6.2 Assignment	19
6.3 Managing sets	20

6.4 The “Default” entry	20
7 Fonts and typography	21
7.1 The fonts that ship	21
7.2 Variation and font set	21
7.3 Managing sets	22
7.4 Adding a font	22
8 Brand: products and company	24
8.1 Company	24
8.2 Products	24
8.3 Uploading logos	24
8.4 Where the values end up	25
Results	26
9 Export	27
9.1 The two routes	27
9.2 What comes out	27
9.3 What differs from the preview	28
9.4 Browsers	28
9.5 Further use	28
10 Saving and moving	29
10.1 Automatically, in the browser	29
10.2 Saving to a file	29
10.3 Loading from a file	30
10.4 Moving to another machine	31
10.5 Resetting everything	31
11 Troubleshooting	32
11.1 The preview stays empty and reports an error	32
11.2 The background switch appears to do nothing	32
11.3 Symbols are missing or look dead	32
11.4 “Browser storage is full”	32
11.5 The export does nothing, or produces something unusable	33
11.6 The title is cut off in the export	33
11.7 Changing a colour has no effect	33
11.8 A font variation cannot be selected	33
11.9 A file cannot be imported	33
11.10 A template has vanished from the selector	34
11.11 The interface is in the wrong language	34
11.12 Nothing helps	34

SECTION

Getting started

IN THIS SECTION	
1 What this tool does	5
2 First steps	8
3 The interface	10

1 What this tool does



The Brand Kit Generator produces **transparent PNG overlays for social media**. An overlay is the graphic that sits on top of a video or a still. Its layout comes entirely from the template – the ones that ship show a few possibilities, a template of your own can be arranged completely differently.

What comes out is a PNG file with a transparent background. It goes into any video editor, streaming program or image editor and sits as the topmost layer over your material. Put a background image under the overlay and that image is exported with it – the file is then finished and ready to upload. The generator does not edit and does not render video – it delivers the graphic.

1.1 The five building blocks

A finished overlay comes from five things, each chosen independently:

Block	What it holds
Product	Title, slogan and logo of a brand or show
Template	The layout – an HTML file whose CSS carries the entire design
Colour set	Seven colour roles and their assignment to design objects
Font set	Named font variations and which one each placeholder uses
Content	Title, subtitle, description and up to two tags of this particular post

The five are combined and rendered in one of four output formats:

product + template + colour set + font set + content → PNG

That is the point of it: every block can be swapped on its own. A different colour set changes the colour without touching the layout. A different template changes the layout without retyping the text.

1.2 The four formats

Format	Size	Name
1:1	1080 × 1080 px	Square
9:16	1080 × 1920 px	Vertical
16:9	1920 × 1080 px	Landscape
4:5	1080 × 1350 px	Portrait

The same template works in all four. It knows which one it is being rendered in and adapts its layout – in a lower third the type grows in the narrow format, because the post is shown smaller in the feed.

1.3 What you need

A browser: Chrome or Edge. The PNG export is verified there. In Safari it is not reliable.

A web server: The application runs in the browser but has to be opened over `http://`, not by double-clicking the file.

Double-clicking does not work

Opened straight from the file manager, `index.html` cannot load its template files – the browser refuses. The preview then stays empty and reports an error. How to start the application properly is in the next chapter.

Nothing else. There is no installation, no build, no account and no server application. Fonts and libraries ship with the project; after the first load everything works offline.

1.4 Language

The interface speaks English and German. It follows what your browser reports and falls back to English. The **DE / EN** button in the top right switches between them at any time; the button shows the language it would switch to.

The switch changes the interface only. Your own names for products, colour sets and font sets stay exactly as you typed them.

1.5 Where your data lives

Everything you set up – company, products, colour sets, font sets, text – stays **in your browser**. Nothing is transmitted, there is no server storing anything.

That has a flip side: clearing your browser data takes your setup with it. How to make a backup is in the chapter *Saving and moving*.

2 First steps

2.1 Starting the application

The application needs a web server. A very simple one that only serves files is enough.

With Visual Studio Code: Install the *Live Server* extension, right-click `src/index.html` in the project, then *Open with Live Server*.

With Python: Open a terminal in the `src/` folder and start:

```
python -m http.server 8000
```

Then open `http://localhost:8000/index.html` in the browser.

💡 How you know it is running

The address bar starts with `http://`, and the preview area on the right shows the overlay on a chequerboard. If it says “The template could not be rendered” instead, the file was opened by double-clicking.

2.2 Your first post in five steps

Everything is filled with sample values on first start. You can begin straight away.

1. Pick a format.

The four formats sit above the preview. Click the one you need – 9:16 for a story, say.

2. Type your text.

The *Creation* area is open on start. Enter your title under *Content for this post*. The preview follows immediately.

3. Pick the look.

Higher up in the same area are four selectors: product, template, font set and colour set. Try a different colour set – the overlay changes colour, the text stays put.

4. Check it against real material.

A row of symbols above the preview switches the background: dark and light chequer-board, black, white and – through the image symbol – your own background image. That

is how you see whether the overlay stays readable over your footage. A background image is more than something to look at: it is exported underneath the overlay.

5. Export.

Bottom right, **PNG** produces the file for the format currently selected. **ZIP (all formats)** puts every selected format into one archive.

Done. The PNG is in your downloads folder and ready for the edit.

2.3 What to set up next

The sample values are there to try things out. For day-to-day use three things are worth doing, in this order:

1. **Company** (*Company* area): name, slogan, copyright line and logo. This rarely changes and appears in nearly every template.
2. **Product** (*Products* area): one entry per show, game or series, with title and logo.
3. **Colour set** (*Colour set* area): put your brand colours into a set of their own instead of reaching for the default every time.

After that a new post is: pick the product, type the title, export.

Everything is saved automatically

There is nothing to confirm. Every change lands in the browser's storage right away and is back after a reload. You only need a file backup when moving your setup to another machine, or to protect it against cleared browser data.

3 The interface

The window has four zones: the header, the sidebar on the left, the preview on the right and the export bar at the bottom.

3.1 Header

Five controls sit in the top right:

Control	Effect
Load	Load a saved setup from a file
Save	Download the current setup as a JSON file
Sun / moon	Switch between the dark and light interface
DE / EN	Switch the interface language
Circular arrow	Reset everything to factory defaults

The language button shows the language it would switch to, not the one you are in – the label is the action, like every other button there.

⚠ Resetting discards everything

The circular arrow deletes your entire setup: company, products, colour sets, font sets and text. It asks once – the first click arms it, a second within four seconds carries it out. Any other click disarms it again.

Switching the interface between light and dark affects the application only. It has no bearing on the overlay or the export.

3.2 Sidebar

A narrow rail on the far left holds seven symbols in three groups:

Post – what changes with every piece

- *Creation*: choosing the blocks and the text of the post
- *Export*: format selection for the ZIP archive and the export buttons

Design – how it looks

- *Template*: pick the layout, see the placeholders, map the typography
- *Font set*: create font variations and set their axes
- *Colour set*: colour values and their assignment

Brand – what rarely changes

- *Products*: products with title, slogan and logo
- *Company*: company details and company logo

3.3 Preview

3.3.1 Toolbar

The four formats sit on the left above the preview. To their right:

Background – five symbols deciding what the overlay sits on in the preview:

Symbol	Background
Grid	Dark chequerboard (default)
Sun	Light chequerboard
Moon	Black
Square	White
Image	Upload your own background image

The chequerboard shows transparency. Black and white check the contrast at both extremes.

3.3.2 Background image

Once an image is uploaded, its framing can be adjusted:

- **Drag** moves the image
- **Scroll wheel** zooms around the pointer, up to six times
- **Double-click**, or a click on the percentage, resets the framing
- The **X** beside it removes the image

The framing is stored as a fraction of the image area, not in pixels. Resize the window and the same part of the image stays in view.

It is kept **per template and per format**, deliberately: a portrait crops a photo differently from a landscape, and a template that lays a scrim along the bottom needs a different part of the picture in frame than one with a bug in the corner. Frame each combination once and it stays that way.

Resetting follows the same rule and only touches what is on screen. Load a **different picture** and every framing is dropped - each was measured against that one photo.

ⓘ The background image is kept

In the export it is drawn underneath the overlay, so the file is finished as it stands. Since 1.2.0 it also survives a reload, and it can be saved with the setup - see [Saving and moving](#).

3.3.3 Badges

Two readouts sit in the upper corners of the preview. On the left the pixel size of the chosen format, with a dot that is green when everything is ready, amber while loading and red on an error. On the right the current combination of product, template and colour set.

3.4 Export bar

At the bottom, the version number and links to the project, the manual and the license sit on the left; the two export buttons on the right. During a ZIP export a progress bar appears in the *Export* area of the sidebar.

SECTION

Working

IN THIS SECTION	
4 Creating a post	14
5 Templates	16
6 Colour sets	19
7 Fonts and typography	21
8 Brand: products and company	24

4 Creating a post

The *Creation* area is the day-to-day workbench. It has two blocks: the choice of building blocks on top, the text below.

4.1 Selection

Four selectors, each with an *Edit* → link that jumps straight to the matching area:

Field	Chooses
Product	Which brand or show
Template	Which layout
Font set	Which font variations
Colour set	Which colours

Under the colour set selector the seven colours of the chosen set appear as small bars. A quick glance to confirm you picked the right one.

The four are independent of each other. Changing the colour set leaves template, product and text untouched.

4.2 Content

Five fields for the piece at hand:

- **Title** – the main statement. The character count sits above the field.
- **Subtitle** – second level, a guest or a subtitle
- **Description** – running text, the smallest text level
- **Tag 1** and **Tag 2** – short markers, usually in capitals, shown as coloured chips

4.2.1 “not used in the template”

Next to a field you may see *not used in the template* in amber. It means the template currently chosen has no place for that value. You can still enter something – it simply is not shown.

That is intent, not a fault. A lower third for video shows a title and a tag; a title card for a feed post also has room for a subtitle and a description. Whatever you typed is kept and reappears the moment you pick a template that uses it.

💡 Which fields a template uses

The *Template* area lists every placeholder under *Placeholders in this template*. The highlighted ones are used, the faint ones are not.

4.2.2 Empty fields disappear

An empty field leaves no gap. Leave *Tag 2* blank and the second chip vanishes entirely – along with its spacing, not as an empty box. The same goes for every text placeholder and for logos.

That lets the same template carry a dense or a sparse post without being adjusted.

4.2.3 Text that is too long

Some templates limit the number of lines, the title to two say. Anything beyond that is cut for the export at a word boundary and given a real ... in the file.

You can see the cut in the preview already. If you do not like the shortened title, shorten it yourself rather than leaving it to the tool.

4.3 The preview keeps up

Every keystroke reaches the preview after about a fortieth of a second. You type and see the result, with nothing to confirm.

What is rendered is always the format currently chosen. Switching format rebuilds it – which is why the status dot in the top left blinks amber for a moment.

5 Templates

A template is the layout of an overlay. Technically it is an ordinary HTML file under `src/templates/` whose entire design sits in its CSS. The application fills in colours, fonts and text – the arrangement is the template’s business.

Something practical follows from that: anyone who knows HTML and CSS can build a template without touching a line of program code. And every template file can be opened on its own in a browser; it then shows sample values and looks just as it does in the application.

5.1 The templates that ship

Template	Layout
Lower Third	The classic full-width bar along the bottom: product logo, brand line with up to two tags, title and copy-right line. For video and stream.
Example 1 – Corner Bug	Reduced to a station bug in the top right: company logo, company name, slogan underneath. Uses no post content and stays readable over any material.
Example 2 – Angled Bar	A bar at the bottom left, only as wide as it needs to be, with the right edge cut at an angle and an accent strip down its left side.
Example 3 – Title Card	A full-frame card for feed posts: tags, a large title, subtitle and description over a scrim that fades up from the bottom, with product logo and copyright on the base line.

The three examples are there to be looked at. They show three very different ways of using the same mechanics.

5.2 Placeholders

The *Placeholders in this template* block lists all twelve placeholders there are. The highlighted ones are used by the chosen template.

Placeholder	Comes from
<code>title</code> , <code>subtitle</code> , <code>description</code> , <code>tag</code> , <code>tag2</code>	<i>Creation</i> area
<code>product.title</code> , <code>product.slogan</code> , <code>product.logo</code>	<i>Products</i> area
<code>company.name</code> , <code>company.slogan</code> , <code>company.copyright</code> , <code>company.logo</code>	<i>Company</i> area

A warning about *unknown placeholders* means the template file uses a name the application does not know – usually a typo in the file.

5.3 Mapping the typography

Below that, every placeholder the template uses gets a row with two fields:

- **Variation** – which font variation from the chosen font set
- **Weight** – which stroke weight. The range depends on the font family; the field names the one in use.

The font size deliberately is **not** here. It stays in the template, because there it is tuned element by element and format by format. In the lower third the title grows by a factor of 1.41 from landscape to portrait while the brand line only grows by 1.20 – so the title dominates more in the narrow format. A single number in the application could not express that.

The division of labour

The **family and its axes** are set in the font set. Which **variation and weight** a placeholder uses is decided here. The **size** stays in the template. Each layer decides what it is best placed to judge.

5.4 Taking a template out of the catalogue

Under the selector sit the file path and, beside it, **Remove**. The button asks once.

Removing deletes no file

A template is a file on disk, and a browser is not allowed to delete files. **Remove** takes the entry out of the catalogue and remembers that. The file stays under `src/templates/` and remains listed in the manifest.

As soon as something is removed, a row appears below listing the removed templates with a **Restore** button that brings them all back at once. *Reset to factory defaults* in the header restores the full catalogue too.

The last remaining template cannot be removed – without a template there would be nothing to render.

5.5 Adding your own template

For anyone who wants to build one:

1. Create an HTML file in `src/templates/`. Easiest is to copy one of the shipped ones and rework it.
2. Add the file name to `src/templates/manifest.json` under `templates`.
3. Reload the page. The template is in the catalogue.

Name, description and category are read by the application from the file's `<title>` and `<meta>` tags – they do not belong in the manifest.

No program code needs touching.

6 Colour sets

A colour set is a named arrangement of seven colours. Switching sets recolours the overlay without touching the layout.

6.1 The seven roles

Colours are not assigned to individual elements but to **roles**. The template decides where a role shows up.

Role	What it is for
Background	Areas, the bar of a lower third say
Accent	Highlights, the product name or a tag
Secondary	Second accent colour, the middle of a gradient
Text	Body text, the title
Subtext	Second text level, the subtitle
Description	Smallest text level
Border	Edges and rules

Seven roles are deliberately few. They force a decision instead of letting a palette sprawl – and they make a colour set work in any template, including one that does not exist yet.

6.2 Assignment

Under the colour values sits the *Assignment* block. It decides **which role a design object uses**:

Object	Default
Tag 1	Accent
Tag 2	Secondary

This creates no eighth colour. The assignment merely points at one of the seven that exist. To colour both tags alike, set both to the same role.

Background is deliberately not offered: the text **inside** a chip runs in the background colour. A chip in the same colour would be invisible.

6.3 Managing sets

Duplicate makes a copy of the set being edited. That is how new sets come about – you never start from nothing.

Show in preview moves the set you are editing into the preview. Editing and showing are separate: you can work on one set while the preview shows another. If the set being edited is already active, it says *Active in preview* instead.

Reset returns a shipped set to its factory state. Sets you created yourself have no factory state; the application says so.

Delete removes a set. Both buttons ask once.

6.4 The “Default” entry

Every list holds an entry named **Default**. It can be neither renamed nor deleted. It is the fallback: if a set is deleted, or a loaded setup names a set this installation does not have, *Default* takes over.

The default colour set is kept neutral and high in contrast so it works over any material. The five named sets beside it are the actual design work.

💡 Setting up your brand colours

Duplicate *Default*, name the copy after your brand and enter the colour values. From then on your set is one click in the *Creation* area – and you never type a hex value again.

7 Fonts and typography

7.1 The fonts that ship

Seven families live in the project. Nothing is fetched from a font service – that keeps the export predictable and means an overlay still looks the same after a vendor updates a family.

Family	Picked for	Axes besides weight
Recursive	WASDCAT corporate typeface	CASL, MONO, slnt, CRSV
Orbitron	Gaming display	–
Archivo	Workhorse with an adjustable width	width 62–125
Inter	Neutral content	–
Space Grotesk	Technical grotesque	–
Fraunces	Editorial serif	SOFT, WONK
Open Sans	The interface font of this application	width 75–100

All seven are **variable fonts**: one file holding a whole design space rather than separate cuts.

7.2 Variation and font set

A **variation** answers *which typeface, tuned how*: a family plus values for its axes. A **font set** is a named group of such variations and works exactly like a colour set – several of them, switchable, saved with the setup.

One font set ships, named *Default*, holding a single variation on Open Sans. Nothing beyond that is prescribed: the typography of a brand is built here, it is not shipped with the application.

7.2.1 Setting axes

Every axis of the chosen family gets a slider. With Recursive, for instance:

- **CASL** fades from technical to handwritten
- **MONO** from proportional to fixed width
- **slnt** slants the face

Change the family of a variation and axes the new one does not know are dropped, while the rest are brought into their valid range.

7.2.2 Why no weight

A variation deliberately carries **neither weight nor size**. Both are hierarchy, and hierarchy belongs to the element, not to the font:

What	Where it is set
Family and axis values	<i>Font set</i> area, per variation
Which variation and weight a placeholder uses	<i>Template</i> area, per placeholder
Size, including the steps per format	CSS of the template

A global weight would either flatten the hierarchy a template builds up, or fight it.

Values are adjusted, not rejected

Enter a weight the chosen family cannot reach and it is brought to the nearest valid one. Open Sans stops at 800 – an entered 900 becomes 800.

7.3 Managing sets

Duplicate copies the font set being edited along with its variations. That is what keeps the placeholder mapping working across sets.

Every font set keeps a variation named *Default* – a duplicated one too. So a placeholder mapping never points at nothing.

Switch font sets and, if the new one does not know a mapped variation, the affected placeholder falls back to *Default* instead of pointing into the void.

7.4 Adding a font

This is a change to the project, not a step in the interface:

1. Put the `.woff2` file into `src/fonts/`
2. Add an `@font-face` block to `src/fonts/fonts.css`
3. Add an entry to `src/fonts.js`, naming under `axes` every axis that should be adjustable
4. Append the license text to `src/fonts/LICENSES.txt`

Step 4 is not a formality: the bundled fonts are under the SIL Open Font License, which requires the license to travel with them.

8 Brand: products and company

These two areas hold what rarely changes. Set up once, you will barely touch them again.

8.1 Company

Four details that appear in nearly every template:

Field	Appears as
Name	<code>company.name</code>
Slogan	<code>company.slogan</code>
Copyright	<code>company.copyright</code>
Logo	<code>company.logo</code>

The copyright line is usually the smallest type in the overlay – a `@ Your Company` in a corner. It is a signature, not a legal department.

8.2 Products

A product is anything with an identity of its own: a show, a game, a series, a format. Each gets a title, a slogan and a logo.

New creates one, **Duplicate** copies the current one – handy for a series that differs only in name.

The *Default* entry can, as everywhere, be neither renamed nor deleted; it is the fallback when a product is removed.

💡 The product title ends up in the file name

On export the product title becomes part of the file name. A telling title saves you sorting your downloads folder later.

8.3 Uploading logos

Both areas take a logo through **Change logo**. Two things happen automatically.

8.3.1 Raster images are scaled down

PNG, JPEG and WebP are reduced to at most 512 × 512 pixels, keeping their aspect ratio. Smaller images are left alone.

The reason is browser storage. It typically holds around 5 MB, and one high-resolution logo embedded as image data can eat several of those on its own. Without scaling, storage would be full after two or three logos – and the whole setup could no longer be saved.

512 pixels is plenty: even in the largest format a logo is rarely displayed above 150 pixels.

8.3.2 SVG files are cleaned

An SVG file is executable code. It can carry scripts, event handlers and references to the outside. Uploaded SVGs are therefore inspected and stripped of everything that is not part of the drawing. A file that cannot sensibly be cleaned is rejected with a message.

For logos SVG is the better choice: scalable to any size at a tiny file size.

Storage full

If the application reports *Browser storage is full*, your setup was **not** saved. Back it up immediately through *Save* and then remove products you no longer need, or particularly large logos.

8.4 Where the values end up

Whether and where a detail appears is entirely the template's decision. *Example 1 – Corner Bug* uses only the company details, *Example 2 – Angled Bar* only the product. Which placeholders a template fills is shown in the *Template* area.

SECTION

Results

IN THIS SECTION	
9 Export	27
10 Saving and moving	29
11 Troubleshooting	32

9 Export

9.1 The two routes

PNG at the bottom right produces a single file in the format currently chosen.

ZIP (all formats) produces an archive with every format ticked in the *Export* area. You can untick individual ones there if you never need landscape, say.

During a ZIP export the *Export* area shows a progress bar naming the format being rendered.

9.2 What comes out

A PNG with a transparent background, made to lie over your stills or clips as an overlay. If a background image has been loaded and placed, it is exported with it – ready to upload, to social media say.

The file has exactly the pixel dimensions of the format, no matter how large the preview happens to be shown. The preview is a scaled-down view; rendering always happens at full size.

9.2.1 File names

Individual PNG files follow the pattern:

```
product_template_colourset_format.png
```

So `sample_show_lower_third_neon_purple_1x1.png`. Everything is lower case, and every character that is neither a letter nor a digit becomes an underscore – the hyphen of a template id and the colon of the format included.

The ZIP archive is named after the product and contains a folder of the same name:

```
sample_show_overlays.zip
├─ sample_show_overlays/
│   ├─ sample_show_lower_third_neon_purple_1x1.png
│   ├─ sample_show_lower_third_neon_purple_9x16.png
│   └─ ...
└─ brand-setup.json
```

9.2.2 brand-setup.json

Every ZIP carries the complete setup the images came from.

That makes an export reproducible. Anyone needing those files again months later loads the enclosed JSON through *Load* and stands exactly where you stood when you exported.

9.3 What differs from the preview

Two things worth knowing.

The preview backgrounds do not travel – an uploaded image does. Chequerboard, black and white are aids for judging contrast; in the export there is transparency in their place. An uploaded background image is the exception: it belongs to the picture and is exported underneath the overlay.

Overlong text is cut. Where a template limits the number of lines, text beyond it is cut at a word boundary before the export and given a real . The browser does not draw its own ellipsis into an image file – without this step the text would simply be chopped off in the PNG.

What you typed is untouched; only the image file is shortened.

9.4 Browsers

Safari is not reliable

The export is verified for Chrome and Edge. Safari handles the underlying technique differently; results can be incomplete or fail entirely. Use Chrome or Edge to export.

9.5 Further use

The PNG goes on top of your material as the uppermost track:

- **Video editors** – as an image in the timeline, above the clip
- **Streaming software** – as an image source above the camera source
- **Image editors** – as a layer above the photo

No conversion is needed. Where the overlay is transparent, that transparency is in the file and is understood directly by all of them.

10 Saving and moving

10.1 Automatically, in the browser

Every change is put into your browser's storage a moment later. After a reload everything is back. Nothing needs confirming.

That storage has three properties worth knowing:

- It is **per browser and per machine**. Chrome at the office and Chrome at home know nothing of each other.
- It is **deleted along with** your browser data – including when you only meant to clear cookies.
- It holds around **5 MB**. That is why logos are scaled down on upload.

Browser storage is not a backup

It is convenient but volatile. Anything you have worked on for a while belongs in a file as well.

10.2 Saving to a file

Save in the header downloads the complete setup as a JSON file.

That file is your backup, your way onto another machine, and your means of keeping several setups side by side – one per client, say.

Not included is the choice between the light and dark interface, or the interface language. Those belong to the workplace, not to the brand.

10.2.1 The background image

With a background loaded, the generator asks what to do with it:

- **With the picture** – one complete file. It grows to the size of the photo and is awkward to send by mail.
- **Without the picture** – a small file. It keeps the framing and remembers the file name. After loading, the preview shows that name; pick the picture and the framing is already in place.

Pick a **different** picture from the one named and the generator asks first. The framing that came with the setup was measured against the named picture and would otherwise go without a word - and with no picture on screen, there is nothing to see it go. *Cancel* keeps both the framing and the expectation, so you can try again.

The framing itself travels either way. It is kept **per template and per format**: a portrait crop sits nowhere near a landscape one, and a template that lays a scrim along the bottom wants a different part of the picture in frame than one with a bug in the corner. Load a **different picture** and every framing is dropped - each was measured against that one photo.

i The picture is not in the 5 MB store

A photo does not fit there, and one oversized value would make the whole save fail. The generator keeps it separately. It is still there after a reload.

10.3 Loading from a file

Load opens the file picker. After choosing, a dialog appears **before** anything is taken over. It shows:

- the file name
- the schema version of the file
- which areas it contains
- a note about the version, where applicable

⚠ Loading replaces the current setup

An import overwrites company, products, colour sets, font sets and the text. Anything not saved to a file is gone afterwards. The dialog says so; *Cancel* or the Escape key back out, *Replace setup* goes ahead.

10.3.1 When a file is rejected

Not every JSON file is a setup. Every file the generator writes carries an identifier. If it is missing, or belongs to another tool, the file is rejected – with a message saying why. You cannot import a foreign configuration by accident.

10.3.2 When the version does not match

A file from a newer version of the generator is still read. The dialog notes that parts this version does not know will be skipped.

The other way round: whatever an older file does not bring – a font set, the typography mapping – is filled in with the defaults on load rather than producing an error.

10.4 Moving to another machine

1. On the old machine, **Save**
2. Transfer the JSON file
3. On the new machine, open the application and **Load**
4. Confirm in the dialog

The template files themselves travel with the project folder, not with the JSON. Anyone who built their own templates has to take them along.

10.5 Resetting everything

The circular arrow in the top right restores the factory state: the shipped products, colour sets and font sets, the sample text and the full template catalogue, including any previously removed.

The button asks once – the first click arms it, a second within four seconds carries it out.

Save first

Make a backup through *Save* before resetting. A reset cannot be undone; a file can simply be loaded again.

11 Troubleshooting

11.1 The preview stays empty and reports an error

Message: *The template could not be rendered*

Most common cause: The file was opened by double-clicking and the address bar starts with `file://`. Browsers refuse to load the template files in that case.

Fix: Open the application through a web server, see *First steps*. The address has to start with `http://`.

Other cause: The template manifest names a file that does not exist, or the file has no element with the class `frame`. The message names the file.

11.2 The background switch appears to do nothing

The switch acts on the area **behind** the overlay. When the preview reports an error, a nearly opaque message sits on top of it – the background does change, but it is covered.

Fix the error message first, see above.

11.3 Symbols are missing or look dead

Opening the application right after the project was updated, the browser may still be using old program files from its cache.

Fix: Reload bypassing the cache – `Ctrl+Shift+R` on Windows, `Cmd+Shift+R` on macOS.

11.4 “Browser storage is full”

Browser storage is full; your setup was **not** saved.

Right away: *Save* in the header, to rescue your work into a file.

Then: Delete products you no longer need, especially ones with a large logo. Store logos as SVG rather than PNG – they are many times smaller.

11.5 The export does nothing, or produces something unusable

Check the browser. The export is verified for Chrome and Edge. In Safari it is not reliable.

Check for a download block. Some browsers block several downloads in a row. A ZIP export is one file; repeated single exports can trip the block.

11.6 The title is cut off in the export

That is intended where the template limits the number of lines. The overhang is cut at a word boundary and given a ...

Fix: Shorten the title. Or pick a template with more room for text – *Example 3 – Title Card* allows three lines.

11.7 Changing a colour has no effect

Editing and showing are separate. You may be editing a different set from the one the preview shows.

Fix: Click **Show in preview** in the *Colour set* area. If it already says *Active in preview*, you are editing the right set – and the template simply does not use that role in the place you are looking at.

11.8 A font variation cannot be selected

The mapping in the *Template* area only offers variations from the font set **currently chosen**.

Fix: Select the right font set in the *Creation* area, or create the variation you want in the current set.

11.9 A file cannot be imported

Message: *No Brand Kit setup or The file belongs to ...*

The file does not carry the Brand Kit Generator's identifier. Either it comes from another tool, or it is not a setup at all – a `brand-setup.json` from a ZIP of a very old build, say.

Fix: Use a file produced by *Save* of this application.

11.10 A template has vanished from the selector

It was taken out of the catalogue with **Remove**. The file is still there.

Fix: Click **Restore** in the note in the *Template* area. Alternatively, *Reset to factory defaults* restores the full catalogue – but resets everything else along with it.

11.11 The interface is in the wrong language

The application follows what your browser reports and falls back to English.

Fix: The **DE / EN** button in the header switches at any time; the choice is remembered. It shows the language it would switch to.

11.12 Nothing helps

Reset as a last resort – but **save first** through *Save*. You can then load the file again and see whether the problem was in your setup or in the build.